

WADF-2026-M01 — Week 3 Lab Walkthrough Guide

Bash Scripting, Hardware Awareness & Storage

This is a guide to help you understand **what to do and why** at each step. You still need to run every command yourself in your own VM, capture your own screenshots, and write your own reflection — that's what's being assessed.

Deadline: 10 July 2026, 23:59 WAT **Submit:** one evidence report (PDF) + the two `.sh` scripts + the three `.txt` outputs, all named with your registration number.

Before you start

1. Open your Ubuntu VM (Desktop or Server) and take a **snapshot** — good habit before any lab work.
2. Create your working folders:

```
bash

mkdir -p ~/wadf-labs/week3/{scripts,output}
```

- `-p` means "create parent folders too, and don't complain if they already exist." This gives you `scripts/` (for your `.sh` files) and `output/` (for your report `.txt` files) in one shot.
3. Keep a terminal open throughout — you'll be screenshotting prompts + output, not just commands.
-

LAB 5: Practical Bash Scripting & System Inventory

5A — Your First Reusable Bash Script

Goal: a script that prints who ran it, when, and where.

1. Move into your scripts folder and open the editor:

```
bash
```

```
cd ~/wadf-labs/week3/scripts
nano system_greeting.sh
```

2. Write the script. Type something like this into nano:

bash

```
#!/usr/bin/env bash
# system_greeting.sh
# Purpose: print a friendly system status line for the current user/host/date
# Author: <your name> - Week 3 Lab 5A

current_user="$(whoami)"
current_host="$(hostname)"
current_date="$(date '+%Y-%m-%d %H:%M:%S')"

echo "Hello, $current_user!"
echo "You are logged into: $current_host"
echo "Current date/time: $current_date"
```

Why each part matters:

- `#!/usr/bin/env bash` (the "shebang") tells the OS which interpreter to run the file with, using whatever `bash` is first on the `PATH` (more portable than hardcoding `/bin/bash`).
 - Lines starting with `#` are comments — they document intent for the marker (and future-you).
 - `$(...)` is **command substitution**: it runs a command and captures its output into a variable.
 - Quoting `"$current_user"` protects against word-splitting if a value ever contains spaces.
3. Save and exit nano: `Ctrl+O`, `Enter`, then `Ctrl+X`.
 4. **Display it with numbered lines** (this is your evidence that the file has the right content):

bash

```
nl -ba system_greeting.sh
```

5. Make it executable and check permissions:

```
bash
```

```
chmod u+x system_greeting.sh  
ls -l system_greeting.sh
```

- `chmod u+x` adds execute permission for the **owner** only (not group/world) — least-privilege habit.
- `ls -l` should show something like `-rwxr--r--` — the `x` in the owner triplet confirms it worked.

6. Run it and save the output to a file, while still seeing it on screen:

```
bash
```

```
./system_greeting.sh | tee ~/wadf-labs/week3/output/greeting_report.txt
```

- `./` is required because your current directory usually isn't on `PATH`.
 - `tee` splits the output: it prints to your terminal *and* writes it to the file simultaneously.
7. **Evidence to screenshot:** the `nl` output, the `ls -l` permissions line, and the `tee` run showing both terminal output and confirming the file was created (you can `cat` the file afterward to double-check).

5B — Inputs, Conditions and Loops

Goal: a script that takes a project name, validates it, and creates 3 note files if valid.

1. Open a new script:

```
bash
```

```
cd ~/wadf-labs/week3/scripts  
nano create_notes.sh
```

2. Write the script:

```
bash
```

```
#!/usr/bin/env bash
# create_notes.sh
# Purpose: create 3 numbered note files for a given project name, with input

read -r -p "Enter project name: " project_name

# Validation: reject empty input or anything containing a slash (path traversal)
if [[ -z "$project_name" || "$project_name" == */* ]]; then
    echo "Error: project name must not be empty or contain '/' characters."
    exit 1
fi

target_dir=~/.wadf-labs/week3/generated-notes/"$project_name"
mkdir -p "$target_dir"

for i in 1 2 3; do
    echo "Notes for project: $project_name" > "$target_dir/note_${i}.txt"
    echo "Created: $target_dir/note_${i}.txt"
done

echo "Done. All notes created in $target_dir"
```

Why each part matters:

- `read -r -p "..."` reads a line of input; `-r` stops backslashes being interpreted (raw mode); `-p` shows a prompt.
- `[[ -z "$project_name" ]]` tests for an empty string. `"$project_name" == */*` tests whether it contains a slash — this is your **input validation**, stopping someone accidentally (or maliciously) writing outside the intended folder.
- `>&2` sends the error message to **stderr**, not stdout — standard practice for error output.
- `exit 1` is a **non-zero exit code**, the Unix convention for "something went wrong." A script that exits `0` on failure is misleading to anything (or anyone) checking its result.
- The `for i in 1 2 3; do ... done` loop is your repeated action — one iteration per note file.
- Quoting `"$target_dir/note_${i}.txt"` avoids word-splitting problems if paths ever contain spaces.

3. Save (`Ctrl+O`), (`Enter`), (`Ctrl+X`), then make executable:

4. **Test success** — run it and give a valid name, e.g. `demo-project`:

```
bash
./create_notes.sh
echo $?
```

`echo $?` prints the exit code of the last command — should show `0` for success. 5. **Test failure safely** — run again and press Enter with no input (or type something with a `/`):

```
bash
./create_notes.sh
echo $?
```

This time you should see your error message and `echo $?` should print `1`. 6. **Verify the results:**

```
bash
find ~/wadf-labs/week3/generated-notes -maxdepth 2 -type f 2>/dev/null | sor
```

Confirm the successful run's files exist, and that the failed run created **nothing**. 7. **Evidence to screenshot:** the script source (`nl -ba create_notes.sh`) if you want numbered lines like in 5A), the successful run + exit code, the failed run + error + exit code, and the final `find` listing.

5C — Read-Only Hardware & OS Inventory

Goal: a plain-text summary of your VM's hardware, produced entirely from read-only commands (nothing here changes system state).

Run each of these and note what it tells you:

Command	What it shows
<code>lscpu head -n 25</code>	CPU architecture, model name, core/thread count
<code>free -h</code>	RAM total/used/free, in human-readable units
<code>lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS</code>	Block devices, their size, whether disk/partition, filesystem type, and where they're mounted
<code>df -hT</code>	Filesystem capacity/usage per mount point, with filesystem type
<code>ip -br link</code>	Network interfaces and their link state (up/down)
<code>ip -br addr</code>	Same interfaces with their IP addresses
<code>systemd-detect-virt 2>/dev/null true</code>	Tells you if you're running in a VM (e.g. <code>kvm</code> , <code>vmware</code> , <code>oracle</code>) or <code>none</code> if bare metal

Build the report — you can pipe each command's output into the file in sequence, e.g.:

```
bash
{
  echo "=== CPU ==="; lscpu | head -n 25
  echo; echo "=== Memory ==="; free -h
  echo; echo "=== Storage ==="; lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
  echo; echo "=== Filesystems ==="; df -hT
  echo; echo "=== Network ==="; ip -br link; ip -br addr
  echo; echo "=== Virtualisation ==="; systemd-detect-virt 2>/dev/null || echo
} | tee ~/wadf-labs/week3/output/system_inventory.txt
```

- The `{ ... }` groups all the commands so a single `tee` captures everything.
- **Virtualisation conclusion:** state clearly in your report whether the system is virtualised, and quote the actual line of evidence (e.g. `systemd-detect-virt` returning `kvm`, or `lscpu` showing a hypervisor vendor line) that supports it.
- **Sanitise before submitting:** don't include real external IP addresses, hostnames tied to non-lab systems, or any tokens/passwords.

Evidence to screenshot: each command's output, plus a look at the final

`system_inventory.txt` (e.g. `cat` it).

LAB 6: Storage Architecture

6A — Map Devices, Partitions and Filesystems

1. List block devices with filesystem info:

```
bash
lsblk -f
```

Shows device → partition hierarchy, filesystem type, and UUID. 2. **Cross-check mounted filesystems:**

```
bash
findmnt
df -hT
```

`findmnt` shows the full mount tree; `df -hT` shows capacity per mounted filesystem — compare them to confirm they agree. 3. **Note the UUID** of your primary (root) filesystem from the `lsblk -f` output — you'll reference this in your map. 4. **Review** `/etc/fstab` **without editing it:**

```
bash
cat /etc/fstab
```

This file defines what gets mounted at boot. You're only *reading* it here. 5. **Optional cross-check:**

```
bash
blkid 2>/dev/null | head -n 20
```

6. Draw your storage map — a simple text or hand-drawn diagram like:

```
/dev/sda (disk)
├─ /dev/sda1 (partition, ext4, UUID=xxxx-xxxx) → mounted at /
└─ /dev/sda2 (partition, swap)
```

Use your *actual* device names/UUIDs from your VM output, not this example.

6B — Locate Common Linux Data Categories

Goal: know where Linux keeps different kinds of data, and prove it with real evidence from your VM.

Category	Typical location	Verify with
User files	<code>/home</code>	<code>ls -ld /home</code> , <code>ls ~</code>
System configuration	<code>/etc</code>	<code>ls -ld /etc</code> , <code>find /etc -maxdepth 1 -type d head</code>
Logs	<code>/var/log</code>	<code>find /var/log -maxdepth 1 -type f 2>/dev/null head</code>
Temporary files	<code>/tmp</code>	<code>ls -ld /tmp</code> , <code>stat /tmp</code>
Installed software	<code>/usr</code> (and <code>/opt</code> for third-party packages)	<code>ls -ld /usr /opt</code>
Runtime state	<code>/run</code>	<code>ls -ld /run</code> , <code>stat /run</code>
Kernel/process info	<code>/proc</code>	<code>cat /proc/uptime</code> , <code>stat /proc</code>

1. Run the whole batch:

```
bash
```

```
ls -ld /home /etc /var /tmp /usr /opt /run /proc /sys
find /var/log -maxdepth 1 -type f 2>/dev/null | head
cat /proc/uptime
stat /tmp /run /proc
```

2. **Persistent vs temporary:** identify at least two locations (typically `/tmp` and `/run`) whose contents are cleared or reset on reboot, versus persistent ones like `/home` and `/etc`.
3. **Explain `/proc` and `/sys`:** these are **virtual filesystems** — they don't store data on disk at all. They're generated live by the kernel to expose running process and

hardware/kernel state, which is why `stat` shows them with unusual sizes (often `0`) and why their content changes instantly as the system changes.

4. Write your findings into:

```
bash
nano ~/wadf-labs/week3/output/data_location_map.txt
```

Fill in a small table/list matching the categories above, each with the path and one line of verification evidence.

Evidence to screenshot: command output for at least seven locations, and the finished worksheet file.

6C — Disk Usage and Capacity Investigation

1. Root and home capacity:

```
bash
df -h / ~
```

2. Size of your whole lab workspace, and its subfolders:

```
bash
du -sh ~/wadf-labs
du -h --max-depth=1 ~/wadf-labs | sort -h
```

- `du -sh` gives one total size.
- `--max-depth=1` breaks it down one folder level, and `sort -h` orders results human-readably from smallest to largest.

3. Five largest files in your workspace:

```
bash
find ~/wadf-labs -type f -printf '%s %p\n' | sort -nr | head -n 5
```

- `-printf '%s %p\n'` prints size in bytes then the path; `sort -nr` sorts numerically, largest first.

4. Hidden files check:

```
bash
```

```
ls -la ~ | head -n 30
```

Compare this against a plain `ls ~` — the `-a` flag reveals dotfiles (like `.bashrc`, `.cache`) that a normal listing hides, which is often where "invisible" disk usage hides. 5. **Write a short, evidence-based recommendation** (2-4 sentences) — e.g., "The `.cache` directory accounts for X MB of hidden usage; since it's regenerable, it would be safe to clear as a first step if space becomes tight." Base it on what you actually found — don't run broad cleanup commands, this task only asks you to *investigate and recommend*.

Evidence to screenshot: `df -h`, both `du` outputs, the largest-files list, the hidden-files comparison, and your written recommendation.

Pulling it all together: Submission

1. Rename your files exactly as required:

- `WADF-2026-M01_Week03_<RegNo>_Evidence.pdf`
- `WADF-2026-M01_Week03_<RegNo>_system_greeting.sh`
- `WADF-2026-M01_Week03_<RegNo>_create_notes.sh`

2. Evidence report (PDF) should include, in order: 5A → 5B → 5C → 6A → 6B → 6C, each with your screenshots and short explanations, plus the contents of `greeting_report.txt`, `system_inventory.txt`, and `data_location_map.txt`.

3. Write your reflection — pick one validation control you added (e.g. the empty-input/slash check in `create_notes.sh`) and explain in a few sentences why it matters and what could go wrong without it.

4. Run through the self-review checklist in the workbook before you submit — especially that screenshots show *results*, not just commands typed.

Good luck — this is a straightforward lab once you work through it command by command. The core skill being tested isn't clever scripting, it's disciplined, safe, and well-documented system administration habits.